

Scenario 4: The locks

Slug: ballard-locks-alert - **Repo:** github.com/joelcapgemini/titan-watch-lab - **Your role:** incident commander, full authority, next page

The kaiju, in thirty seconds

- **Skarnyx.** Brine Serpent, submerged, 88 m, 55 km/h. Target Seattle. Drawn to fresh water; will try to pass the Ballard Locks, the gate between the Sound and the lakes. If the gates go, the lakes drop and the salt comes in.
- **There is no clock.** That is the problem. A sonar log shows a submerged track in the Sound, nothing has surfaced, and it can stay down for hours.
- **A return is not a sighting.** Whale, submarine or Skarnyx; nobody can tell from one, or say how sure they are.
- **Who sees it.** Titan Watch sees a return, sometimes. The lockmaster sees gate strain. The Coast Guard sees the surface. Nobody sees all three. That gap is the scenario.

Seattle, Kirkland and Bellevue share the lake shoreline. Alert all three on a sonar log alone, or wait for something to surface and lose the lakes. **Your job:** get enough evidence fast enough to pick the smallest response that works, from closing the gates up to a three-city alert, and record why.

Outcome, same for every scenario

Save as much human life as possible, then prevent as much public property damage as possible, in that order. A software tool is expected to come out of this.

You are graded on, in this order: people killed or injured on the three waterfronts; the lakes, meaning whether the lock gates hold; whether the alert, if raised, is believed and acted on; false alarms, each one costs the next alert; and whether every decision you made was a decision, recorded, with a name on it.

The coach interviews you against this, narrows it to this gate, and restates it after synthesis. The original stays in the scope file beside the restatement.

Who reports to you

Nine specialists have already been interviewed. Their interviews are in `.copilot-tracking/dt/ballard-locks-alert/`. Do not read them. The coach has, and will quiz you on what they said.

Reports to you	What they own	What they keep saying
Army Corps lockmaster	The gates	Twenty minutes to close; nobody has asked
Titan Watch sonar analyst	The returns	"Sixty percent, my guess, not ours"
Titan Watch duty analyst	The threat board	No row for "submerged, somewhere"
Seattle duty officer	Seattle shore	Wants two sources that agree
Kirkland emergency manager	East shore, Bellevue	Hears last; wants to act small
County alert officer	The wireless alert	All or nothing; opt-outs are permanent
Coast Guard sector	The surface	One boat, no photograph, "four radios"
Fisheries biologist	Marine life	Can rule out a whale; not asked
Houseboat resident	One floating home	Trusts a closed gate over an alert

Already done, do not redo

Scope, stakeholder map, assumptions log, world rules, nine interviews, security scoping answers. All under `.copilot-tracking/dt/ballard-locks-alert/`. The coach reads them.

Out of scope for everyone: engaging Skarnyx, anything past the waterfronts or after a breach.

Your card: Incident commander, Ballard Locks

Scenario 4, the locks. Kaiju facts are on page 1 of this pack. Human facts are yours: invent and label.

Outcome: save as much human life as possible, then prevent as much public property damage as possible, in that order. The coach narrows it; that is fine.

Slug: ballard-locks-alert-commander

Who you are

You are in charge. The Army Corps, Titan Watch, the Coast Guard, the county and three cities answer to you. You can close the locks, raise or hold an alert at any level in Seattle, Kirkland and Bellevue, and task sonar and the patrols. Nobody will argue. The only things you do not control are Skarnyx and the next return.

Who you can call, and for what

- Lockmaster: gate strain, time and cost to close.
- Titan Watch analysts: the last return, the board.
- Seattle duty officer: the Seattle shore.
- Kirkland: the east shore, Bellevue by proxy.
- County alert officer: what an alert or a false one costs.
- Coast Guard: the Sound's surface, one boat.
- Biologist: could it be a whale.
- Harbour patrol: the lake side, the docks.

Each sees a piece. You see what they tell you, when they tell you. No screen of your own.

Your scope

In scope, you decide and order:

- Close the locks, yes or no, and when.
- Alert or hold, at what level, which cities, on what evidence.
- Where sonar and the patrols look next.

Out of scope:

- How a specialist carries out your order. They know their job.
- Where Skarnyx is or goes. Titan Watch reports it; nobody controls it.

Anchors, fixed

- There is no clock. Nobody will tell you how long you have. Every wait costs; so does every alert.
- Every piece of evidence arrives through a different person on a different schedule, none with a confidence.

Three phrases

"I do not know." - "Assume and label it." - "Ask the specialist, then tell me."

Next

Turn the page. The run sheet has every command and prompt for this scenario, in order.

How to proceed: Ballard Locks

Order of work, with every prompt filled in for this scenario. Full detail is in the shared instructions.

Lab 1: Discovery

- 1 /clear, then /hve-core:dt-coach. It lists the projects it can resume. Pick yours, the one named on page 1. Then type: I am the incident commander. Start the interview. If no list appears, type resume project <name on page 1> first.
- 1 Answer five questions as the commander. Read your interview file once.
- 2 Same session, paste:

Project ballard-locks-alert. Read the shared scope under .copilot-tracking/dt/ballard-locks-alert/, every interview there, and my interview under .copilot-tracking/dt/ballard-locks-alert-commander/. Restate the outcome in light of the interviews, keeping the original beside it. Summarise the problem as if hearing it for the first time. List what you still need before a PRD. Then draft the PRD with non-functional requirements, close gaps with labelled assumptions, and state what it rests on. Then a technology-neutral diagram as Mermaid. Then an opinionated Azure overlay with the minimum services, rejected alternatives recorded, and a latency budget. Write everything under .copilot-tracking/dt/ballard-locks-alert/. Run without stopping. Label all interviews role-play.
- 1 PRD, diagram and overlay land under dt/ballard-locks-alert/.

Lab 2: Security and Responsible AI

Write three predicted security findings on paper first. Two terminals.

- 1 Terminal 1: `/clear`, then `/hve-core:security-planner`. Paste:

Use the security planner to plan the production security for this solution. PRD and architecture are under `.copilot-tracking/dt/ballard-locks-alert/`. Scoping answers are in `security-scoping-answers.md` in the same folder; treat every row as answered and labelled, do not ask them again. Threat-model the boundaries, map controls to OWASP, NIST and CIS, produce a backlog. Ask me only about severe or blocking issues. Run all phases without stopping. Prioritise by severity. Do not commit or push.

- 1 Terminal 2: `/clear`, then `/hve-core:rai-planner-agent`. Paste:

Run an RAI assessment on the single-save decision: the disaster-recovery AI in this app's Last Stand scene that recommends which one of two cities to save when only one can be saved. Today it outputs a single verdict with no confidence, no explanation and no audit trail, and a commander approves it under time pressure. Assess it against NIST AI RMF and treat the autonomous life-safety decision as the central risk. Use project name `single-save-decision`. Imagine the scoping answers, label them, run to completion unless blocking.

- 1 When the security planner finishes: "Plan summary by severity: counts per level, Critical items one line each, recommended next planner." Compare with your paper. Read the RAI go/no-go.

Lab 3: Research, Plan, Implement

- 1 Open your PRD under `dt/ballard-locks-alert/`. Pick one functional requirement the app could carry. One screen, one rule, one feed.
- 2 `/clear`, then `/hve-core:rpi-agent`. Ask: "What do you need from me to create an RPI plan?" Then paste, filled in:

Goal: implement <requirement, one sentence> from the PRD at `.copilot-tracking/dt/ballard-locks-alert/<prd file>`.
Scope: <one component or module>. Nothing else.
Acceptance criteria: <two or three testable lines>.
Constraints: no backend, no tracker, no cloud tenant; mock or defer and log the gap. No tracking paths in code or commits.
Validation: the existing test, lint and build scripts.
Context: the PRD and the Azure overlay in the same folder.
Mode: research, stop for my review; plan, stop for my approval.

- 1 Research stops: read it, fix the file if wrong, then `/hve-core:rpi-plan`.
- 2 Plan stops: amend one item in the plan file, tell the agent "plan amended in the file".
- 3 Homework: `/hve-core:rpi-implement`, then `/hve-core:rpi-review`.